

TEAM 1 — **Achyuth , Vishnu, Rohit , Vismay**

Oracle VirtualBox (Virtualization Fundamentals)

Instructions:

1. Install VirtualBox on one member's laptop. Download a free lightweight OS ISO (e.g., Ubuntu Desktop or Lubuntu).
2. Create a new Virtual Machine (VM): assign it virtual CPU cores, virtual RAM, and a virtual hard disk. **Screenshot every setting screen** — this is your "CPU virtualization" and "Memory virtualization" evidence.
3. Install the guest OS inside the VM and boot it.
4. Go to **Settings** → **System** → **Processor** and change the number of virtual CPUs; note how VirtualBox maps this to the host's physical cores (CPU virtualization).
5. Go to **Settings** → **Storage** and attach a virtual optical drive and virtual hard disk; explain how the hypervisor intercepts and emulates real I/O devices (I/O virtualization).
6. Enable **Shared Folders** and **Clipboard sharing** between host and guest — this demonstrates desktop virtualization concepts (a "virtual desktop" experience).
7. Create a **snapshot** of the VM, then break something inside the VM (delete a file), then restore the snapshot. Explain this in terms of virtualization enabling flexible resource/data management.
8. Optionally clone the VM to simulate a very small "virtual cluster" (2 VMs running together, each with allocated resources).

Report must explain: Type-1 vs Type-2 hypervisors (VirtualBox is Type-2), the 5 implementation levels of virtualization (ISA, HAL, OS, library, application level — identify which level VirtualBox operates at and why), and how this relates to virtual clusters and resource management in real data centers.

TEAM 2 — **Sahal, Sherin, Aman, Nandakishore**

AWS Free Tier (IaaS & Deployment Models)

Instructions:

1. Create a free-tier AWS account.
2. Launch an **EC2 t2.micro instance** (free tier eligible) — choose Amazon Linux or Ubuntu. This is raw **Infrastructure as a Service**: you're renting a virtual machine, not an application.
3. Note during launch: choice of instance type (= renting compute), storage (EBS volume, = renting storage), and security group (= renting network config). Screenshot each.
4. Connect to the instance via SSH (or EC2 Instance Connect in-browser) and run a simple command (sudo apt update) to prove it's a real usable machine.
5. Create an **S3 bucket** (free tier) and upload a file — this is storage-as-a-service.
6. In IAM, create a restricted user and explain how AWS enforces the shared responsibility model.
7. Terminate/stop the instance at the end to avoid charges.

Report must explain: How EC2 = IaaS (compare with PaaS/SaaS conceptually, even without using them), which of the 4 deployment models (public/private/hybrid/community) AWS itself represents by default, and how a company could turn this same AWS account into a "private cloud" using AWS VPC + dedicated instances, or a hybrid setup using AWS Outposts (research this, no need to implement).

TEAM 4 — Sreedeeep, Sreenanda, Devika Surendran, Fayha

Microsoft Azure Free Account (SaaS, Hybrid & Service Management)

Instructions:

1. Create a free Azure account (free credit + always-free services).
2. Use a ready-made **SaaS product** first: log into **Microsoft 365 / Office Online** (free with the account) and co-edit a Word doc with a teammate simultaneously — this IS Software as a Service (no install, browser-based, multi-user).
3. Then go to Azure Portal and create an **Azure App Service** (PaaS) web app from a template, to contrast with the pure SaaS experience above.
4. Explore **Azure Arc** or **Azure Hybrid Benefit** documentation (Microsoft's official hybrid-cloud offering) — you don't need to deploy it, but take screenshots of the official docs page and summarize how Azure lets a company connect on-premise (private) servers with Azure's public cloud, i.e., a real **hybrid cloud**.
5. Look at **Azure Government / Azure for Nonprofits** pages as a real-world example of **community cloud** (a cloud shared by organizations with common concerns, e.g., government agencies).
6. Delete all created resources at the end.

Report must explain: SaaS vs PaaS with your two hands-on examples, all four deployment models using Azure's real product lines as concrete evidence (public = normal Azure, private = Azure Stack, hybrid = Azure Arc, community = Azure Government), and what "service management in cloud computing" (SLAs, billing dashboards, service health monitors — check Azure Service Health in the portal) means in practice.

TEAM 5 — Shad, Shaza, Devika V, Safwan

Docker Desktop (Levels of Virtualization, VMs vs Containers)

Instructions:

1. Install Docker Desktop.
2. Run `docker run hello-world` in a terminal — screenshot the output.
3. Pull and run a real lightweight app: `docker run -d -p 8080:80 nginx`, then open `localhost:8080` in a browser to see the nginx welcome page running **inside a container**.
4. Run `docker ps` and `docker stats` to see the container's isolated CPU/memory usage — this is OS-level virtualization (compare explicitly with Team 1's VirtualBox, which is hardware-level virtualization).

5. Create a second container from a different image (e.g., `docker run -d -p 8081:80 httpd`) so two "virtual servers" run simultaneously on one physical machine — this is a **mini virtual cluster**.
6. Write a very simple Dockerfile (5 lines) for a basic app and build/run it with `docker build` and `docker run`, to show how virtualization enables reproducible, portable deployment (a data-center automation concept).
7. Stop and remove all containers at the end (`docker stop`, `docker rm`).

Report must explain: The difference between hardware-level virtualization (Team 1's VMs, full OS each) and OS-level/container virtualization (yours, shared kernel, lightweight) — this maps directly to "implementation levels of virtualization" in the syllabus. Also explain how container clusters (conceptually, like what Team 6 will show with Kubernetes) enable data center automation.

TEAM 6 — Abhishek, Sreehari, Devang, Arjun

Killercoda - Virtual Clusters & Data Center Automation)

Instructions:

1. Open the Killercoda Kubernetes playground and start any beginner scenario (e.g., "Kubernetes Playground" or "Getting Started").
2. Run `kubectl get nodes` — observe that Kubernetes is managing a **cluster** of virtual nodes, not a single machine.
3. Deploy a sample app: `kubectl create deployment demo --image=nginx` then `kubectl get pods` and `kubectl scale deployment demo --replicas=3`. Watch 3 identical containers get created automatically — this is **virtual cluster resource management** in action.
4. Run `kubectl delete pod <one-pod-name>` and immediately run `kubectl get pods` again — Kubernetes will have already restarted a replacement pod automatically. Screenshot this. This is **data center automation**: self-healing infrastructure with no human intervention.
5. Run `kubectl describe node` to see how CPU/memory resources of the cluster are tracked and allocated — this is resource management at cluster scale.
6. Try `kubectl scale deployment demo --replicas=0` then back to `--replicas=5` to show elastic scaling.

Report must explain: What a "virtual cluster" is and how it differs from a single VM (Team 1) or a single container, how Kubernetes automates tasks that a human data-center admin used to do manually (this is literally "virtualization for data center automation" in the syllabus), and how this relates to the resource-management module.

TEAM 7 — Lamih, Fadiya, Prinsha, Drupath

Google Workspace (SaaS, Security-as-a-Service, Pros/Cons)

Instructions:

1. Use **Gmail + Google Drive + Google Docs** (free Google account is enough, trial not even required) — collaboratively create a shared doc, comment on it live as a team, and share a Drive folder with view-only vs edit permissions. This is a full SaaS experience: no installation, subscription/account-based, always-updated, accessible from anywhere.
2. Sign up for **Cloudflare's free plan** and add a placeholder domain (or read their free plan feature list on the pricing page) — Cloudflare's DDoS protection, firewall, and SSL are real examples of **Security as a Service (SECaaS)**: security delivered as a cloud subscription instead of installed hardware/software.
3. Check <https://haveibeenpwned.com/> with a (throwaway/test) email to understand cloud-based breach-monitoring as another SECaaS example (do this only for awareness/discussion, not for real personal accounts).
4. As a team, list every permission Google Drive asked you to grant, and every setting Cloudflare offers for free vs paid — use this as raw material to build a genuine **pros vs cons of cloud computing** table (pros: no install, auto-updates, accessible anywhere, pay-as-you-go, security offloaded to experts; cons: needs internet, data privacy/trust concerns, vendor lock-in, recurring cost, limited customization on free tiers).

Report must explain: SaaS in depth (your primary hands-on topic), Security as a Service as a distinct EaaS category alongside IaaS/PaaS/SaaS, and a genuinely evidence-based pros/cons list of cloud computing built from what you personally experienced (not copied from a textbook).